TRABALHO DE CONCLUSÃO DE CURSO

Guia de estudos do detector neural de imagens

Fundamentos, implementação, evolução experimental e interpretação dos resultados

Fases implementadas: 1 a 3.9
Material de estudo técnico e explicativo
12 de novembro de 2026

Este material descreve **o sistema implementado até a Fase 3.9**. Seu objetivo é permitir que o grupo compreenda a teoria, leia as escolhas feitas no código e interprete os resultados sem confundir desempenho interno com generalização. A tarefa do detector é classificar a **origem visual estimada** de uma imagem, não verificar se a cena retratada é verdadeira.

Como ler as evidências. O código e as saídas preservadas nos notebooks são a referência principal. Os checkpoints e os resultados das fases recentes estão no Google Drive, que não está montado nesta revisão local; para essas fases, os números abaixo correspondem às saídas do Colab registradas nos notebooks e nos relatórios do repositório. O guia anterior anexado, *Guia de estudo — detector de imagens sintéticas-1.pdf*, descrevia o estado até a 3.4 e foi usado como referência didática, mas seus trechos que diziam que a 3.4 v2 ainda não havia sido treinada estão superados pelos registros posteriores. Nenhuma métrica deste guia foi produzida por novo treinamento.

Sumário

1. Introdução e contextualização
2. Fundamentos de inteligência artificial e redes neurais
3. Arquiteturas e componentes dos modelos
4. Bases de dados e preparação das imagens
5. Funcionamento completo do treinamento
6. Evolução histórica das versões
7. Avaliação e interpretação dos resultados
8. Funcionamento do sistema implementado
9. Decisões técnicas, dificuldades e soluções
10. Limitações e interpretação crítica
11. Glossário técnico
12. Revisão e fixação do conhecimento

1. Introdução e contextualização

1.1. A pergunta do TCC

O problema é estimar se os pixels de uma imagem vieram de uma captura fotográfica ou de geração/alteração por inteligência artificial. No projeto, **real é a classe 0** e **sintética ou alterada por IA é a classe 1**. Essa definição aparece no remapeamento de rótulos da Fase 1 [Notebooks/fase_1/fase1_vit.ipynb], no treinador da 3.9 [experimentos/fase3_9_tgif_finetune.py] e na camada de linguagem das predições [experimentos/linguagem_predicao.py]. Uma foto de uma cena encenada continua sendo fotográfica para esta tarefa; uma imagem de IA que represente um acontecimento plausível continua sintética. O modelo não decide a veracidade de uma notícia.

Modelos gerativos aprendem regularidades estatísticas de imagens e produzem novos arranjos de pixels. Em alguns conjuntos deste TCC, a classe sintética é uma imagem criada por um gerador; em outros, como os pares TGIF, é uma fotografia original **regenerada** por um modelo. A fronteira visual entre classes é difícil

porque geradores podem reproduzir iluminação e textura de câmeras, enquanto fotos reais podem perder detalhes por edição, redimensionamento e compressão. Uma pista que parece útil em um conjunto pode ser característica do formato do arquivo ou da fonte, e não da origem da imagem.

O objetivo experimental documentado é **pelo menos 85% de acurácia em imagens externas independentes**. Isso não foi demonstrado pelos resultados disponíveis. O melhor valor informado no benchmark comparativo Mirage de 2.000 imagens é **62,3% na Fase 3.7**; a versão implementada mais recente, 3.9, obteve **58,75%** nesse mesmo conjunto. A precisão da classe sintética, que tem outra definição, não substitui essa meta. Os detalhes e as ressalvas de comparação estão no estado do projeto [ESTADO_PROJETO.md] e nos capítulos 6–7.

O escopo implementado é um **classificador de imagens** com scripts de coleta/preparo, treinamento, auditoria e avaliação, notebooks que operam no Colab e uma função de apresentação textual da saída. Não há API web de produção neste repositório. Tampouco há um detector de vídeo, prova criptográfica de procedência, localização de regiões editadas ou verificador de coerência física explícito. A ausência dessas funções delimita o que a saída pode afirmar.

1.2. Onde o projeto se situa na IA

Inteligência artificial é o campo amplo de sistemas que executam tarefas associadas a percepção, decisão ou geração. *Aprendizado de máquina* é a parte em que o comportamento é ajustado a partir de dados; *aprendizado profundo* usa redes com várias camadas de parâmetros; *visão computacional* aplica esses métodos a imagens e vídeos. Este TCC combina os três últimos: fornece exemplos rotulados de fotos reais e imagens sintéticas e ajusta uma rede para produzir uma pontuação para cada classe. A regra não é uma lista escrita à mão de defeitos de IA; é uma função aprendida com exemplos, sujeita aos vieses deles.

2. Fundamentos de inteligência artificial e redes neurais

2.1. Aprendizado supervisionado, neurônios e parâmetros

No aprendizado **supervisionado**, cada exemplo de treino tem uma entrada `x` (a imagem transformada em números) e um rótulo `y` (0 ou 1). A rede calcula uma saída `fθ(x)`; o conjunto `θ` contém os parâmetros que mudam durante o treino. A comparação entre a saída e o rótulo gera uma perda. Repetir esse processo com muitas imagens ajusta `θ` para reduzir a perda média. Os datasets PyTorch da Fase 1 [Notebooks/fase_1/fase1_vit.ipynb] e da Fase 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] entregam pares desse tipo.

Um neurônio artificial simples calcula $z = w_1x_1 + ... + w_nx_n + b$: `x` são entradas, `w` são **pesos**, `b` é o **viés** e `z` é um valor intermediário. Uma função de ativação, como ReLU ou GELU, transforma `z` de forma não linear. Sem não linearidade, empilhar camadas lineares seria equivalente a uma única transformação linear. Neste projeto, pesos aparecem nas convoluções, projeções, atenção e classificadores; vieses aparecem nas camadas que os incluem. **Parâmetro treinável** é um valor que o otimizador pode alterar. Uma camada pode estar presente e temporariamente congelada: na primeira etapa da Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb], os ramos herdados da Fase 1 não recebem atualização enquanto os componentes novos aprendem.

Camada é uma transformação com ou sem parâmetros. Por exemplo, `Linear(192, 256)` aprende a converter um vetor de 192 números em outro de 256; `LayerNorm` estabiliza a escala das características; `Dropout` desativa aleatoriamente ativações durante o treino, mas não na inferência. Uma rede não compreende a categoria “real” como um conceito humano: aprende associações entre características e rótulos observados.

2.2. Forward, perda, gradiente e otimização

A **propagação direta** (*forward pass*) percorre as camadas desde os pixels até dois valores finais, os *logits*. A perda de entropia cruzada compara esses logits com o rótulo correto. Para duas classes, sua intuição é penalizar fortemente uma previsão que dá pouca massa à classe verdadeira. Se a probabilidade atribuída à classe correta fosse `0,9`, a penalidade seria menor do que se fosse `0,1`; a implementação usa os logits diretamente, com estabilidade numérica, em `torch.nn.CrossEntropyLoss` (Fase 1 [Notebooks/fase_1/fase1_vit.ipynb], Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb], treinador reaproveitado da 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb]).

A **retropropagação** aplica a regra da cadeia para obter, para cada parâmetro, o gradiente `∂L/∂θ`: quanto uma pequena mudança nesse parâmetro alteraria a perda. O **otimizador** usa os gradientes para atualizar os parâmetros; neste projeto, o algoritmo é AdamW. A **taxa de aprendizado** controla a magnitude das atualizações. Taxa muito alta pode tornar o treino instável; muito baixa pode fazê-lo avançar lentamente. Essa explicação descreve o mecanismo, não diagnostica sozinha uma falha observada no TCC.

Um **batch** é um pequeno conjunto de imagens processado em uma atualização. Uma **iteração** é normalmente uma atualização por batch. Uma **época** é uma passagem planejada pelo número de amostras do treino; com `WeightedRandomSampler(replacement=True)`, como nas fases recentes, uma época sorteia `len(tr_rows)` índices **com reposição**, então algumas imagens podem aparecer mais de uma vez e outras nenhuma vez naquela época. Por exemplo, 3.594 amostras e batch 32 significam aproximadamente 112 batches completos se `drop_last=True`, não 3.594 atualizações. Ver o sampler da 3.4 v2 [experimentos/amostragem_fase3_4.py] e o DataLoader da 3.9 [experimentos/fase3_9_tgif_finetune.py].

2.3. Generalização, overfitting e divisões

Generalizar é acertar imagens não usadas para ajustar o modelo, inclusive de conteúdos, fontes e geradores diferentes. **Overfitting** ocorre quando a rede aprende detalhes específicos do treino e perde desempenho fora dele. **Underfitting** ocorre quando não aprende nem os padrões úteis do conjunto de treino. Não se diagnostica nenhum dos dois por uma única métrica: é preciso comparar curvas e divisões. A distância entre a seleção interna da 3.9 e o Mirage é uma evidência de transferência insuficiente para esse benchmark, mas não identifica uma causa isolada.

O fluxo das fases recentes separa quatro papéis: **treino** altera pesos por gradientes; **seleção interna** escolhe a melhor época, pela AUC; **calibração interna** ajusta temperatura e limiar, depois de escolhido o checkpoint; **teste** mede o resultado com as decisões fixadas. O conjunto OpenFake reservado aparece nos scripts recentes apenas na verificação de separação por hash; não foi inferido nas Fases 3.5–3.9. O Mirage foi avaliado depois do treino de cada fase, mas já era conhecido no desenvolvimento e serve como comparação descritiva, não como teste cego de novas decisões. O papel de cada divisão é visível no treinador da 3.9 [experimentos/fase3_9_tgif_finetune.py] e no guia metodológico [docs/GUIA_METODOLOGIA_TCC.md].

3. Arquiteturas e componentes dos modelos

3.1. Entrada numérica e convoluções

Uma imagem RGB é uma matriz `altura × largura × 3`. O código a converte em tensor PyTorch no formato `batch × canais × altura × largura`. Depois de `ToTensor()`, os canais ficam aproximadamente entre 0 e 1; `Normalize(mean=0,5, std=0,5)` os leva aproximadamente a `[-1, 1]`. O treinamento e a inferência precisam aplicar a mesma convenção de entrada. A Fase 1 [Notebooks/fase_1/fase1_vit.ipynb] e a Fase 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] definem essas transformações.

Uma **convolução** aplica pequenos filtros aprendidos em várias posições. Assim, o mesmo filtro pode responder a bordas e texturas independentemente da posição exata. Camadas sucessivas combinam padrões locais em representações mais abstratas. A vantagem é reutilizar filtros e incorporar a hipótese de que detalhes próximos importam; uma limitação é que relações distantes entre regiões podem exigir várias etapas ou outro mecanismo. No projeto, há convoluções dentro do ramo FFT, na projeção inicial do ViT e em todo o EfficientNet-B0. Isso não significa que cada ramo veja a mesma representação: o ramo FFT recebe o espectro, os outros recebem pixels normalizados.

3.2. ViT-Tiny: regiões, atenção e contexto

O **Vision Transformer** divide a imagem em *patches*. Na implementação da Fase 1 [Notebooks/fase_1/fase1_vit.ipynb] e nas classes copiadas da Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb], um recorte de `224 × 224` é dividido em quadrados de `16 × 16`: são `(224/16)² = 196` patches. Uma `Conv2d` com kernel e passo 16 converte cada patch num vetor de **192 componentes**. O código acrescenta um token aprendível `[CLS]` e posições aprendíveis; assim, a sequência tem 197 tokens.

Em cada bloco de **atenção de múltiplas cabeças**, três projeções lineares produzem *queries* (`Q`), *keys* (`K`) e *values* (`V`). A intensidade de relação entre dois tokens vem de `softmax(QKᵀ/√d)`, onde `d` é a dimensão de cada cabeça. Uma combinação ponderada dos `V` atualiza cada token. O código usa **3 cabeças** e **12 blocos**; cada bloco tem `LayerNorm`, atenção, conexões residuais, outra normalização e um MLP com GELU. A saída usada na fusão é o vetor do token `[CLS]`, de dimensão 192. A atenção permite relacionar regiões distantes, mas seu peso não é uma explicação causal automática do que o modelo "viu"; tampouco garante que ele confira física da cena.

O ViT-Tiny foi **implementado no próprio notebook da Fase 1**, e não carregado como ViT pré-treinado. Seu custo e necessidade de dados ajudaram a motivar warmup, mais épocas e regularização naquela fase, segundo o comentário do próprio treinamento da Fase 1 [Notebooks/fase_1/fase1_vit.ipynb]. A Fase 2 reutiliza os pesos dos ramos ViT e FFT da Fase 1, quando o checkpoint está disponível; as versões 3.4–3.9 começam de um checkpoint completo da Fase 3.3.

3.3. FFT: frequência espacial e ramo aprendido

Frequência espacial descreve a rapidez com que intensidades variam no espaço. Uma área de cor quase uniforme concentra variações lentas; bordas finas e texturas repetitivas geram variações mais rápidas. A **Transformada de Fourier bidimensional** decompõe o padrão espacial em componentes de diferentes frequências e direções. A FFT é um algoritmo eficiente para calculá-la; não é, por si, um classificador.

O método `FFTBranch.to_log_magnitude_spectrum` calcula, por canal RGB, `fft2(x, norm="ortho")`, desloca as baixas frequências para o centro com `fftshift`, toma a magnitude com `abs` e comprime a escala dinâmica com `log1p`. Em seguida, **três convoluções com BatchNorm e ReLU**, *pooling* global e uma camada linear transformam o espectro em **128 características** (classe `FFTBranch` da Fase 1 [Notebooks/fase_1/fase1_vit.ipynb]; mesma arquitetura na Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb]). O logaritmo evita que picos de magnitude dominem numericamente a entrada do ramo. Tomar apenas a magnitude descarta a fase complexa: esse ramo não reconstrói a imagem nem fornece prova da origem. Compressão JPEG, redimensionamento e nitidez também afetam o espectro; por isso, um padrão de frequência não deve ser tratado automaticamente como marca de IA.

3.4. EfficientNet-B0 e a fusão por gate

A **EfficientNet-B0** é uma CNN que combina camadas convolucionais em diferentes profundidades. A Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb] a instancia pelo `torchvision.models.efficientnet_b0` com pesos `IMAGENET1K_V1`. A camada classificadora original não é usada: `m.features` e *pooling* global produzem um vetor de **1.280 componentes**. Como o tensor do projeto está normalizado para `[-1,1]`, `CNNBranch.forward` primeiro

o converte à normalização esperada pelos pesos ImageNet. Isso evita aplicar pesos pré-treinados a uma escala de entrada incompatível. O pré-treinamento fornece características visuais iniciais gerais; não é treinamento prévio específico de detecção de IA.

Na Fase 1, a **fusão é concatenação**: o vetor ViT de 192 componentes é unido ao vetor FFT de 128, gerando 320 entradas para o classificador (`Phase1Detector`). Na Fase 2 e nas seguintes, `Phase2Detector` projeta cada um dos três vetores para **256 componentes**. O **gate** lê a concatenação dessas três projeções, calcula três pesos com `softmax` e faz a soma ponderada dos vetores projetados antes da classificação. Em notação compacta, $h = \Sigma\, g_i\, h_i$, com $g_i \geq 0$ e $\Sigma g_i = 1$. Os pesos variam conforme a entrada. O classificador final devolve **dois logits**, um para cada classe. O código oferece opção de fusão por concatenação para experimentos, mas os checkpoints usados no percurso principal registram `fusion="gate"`. A combinação dá ao modelo acesso a contexto, textura local e espectro; **não há ablação suficiente nos resultados deste guia para atribuir causalmente um ganho a cada ramo**.

Um percurso concreto é: foto RGB → recorte 224 → normalização → ViT (192 números), FFT-CNN (128) e EfficientNet-B0 (1.280) → três projeções para 256 → pesos do gate → vetor combinado → dois logits. A arquitetura da Fase 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] preserva os nomes dos módulos da Fase 2 para aceitar seus checkpoints. Nos scripts recentes, uma mensagem de log “Arquitetura da Fase 2 montada” descreve **a estrutura**; a linha posterior “Pesos iniciais da Fase 3.3 carregados” descreve **os pesos ativos**. São fatos distintos (treinador da 3.9 [experimentos/fase3_9_tgif_finetune.py]).

4. Bases de dados e preparação das imagens

4.1. Dados internos usados para aprender

Fonte	Papel comprovado	Organização e cuidado
Faces do Kaggle + fotos COCO	Treino inicial da Fase 1 e da Fase 2	A Fase 1 [Notebooks/fase_1/fase1_vit.ipynb] inclui faces reais/sintéticas e até 5.000 COCO reais **somente no treino**; seu notebook registra 105.000 treino, 20.000 validação e 20.000 teste. A Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb] registra 85.000 treino, 10.000 seleção, 10.000 calibração e 20.000 teste. São protocolos próprios, não teste de generalização universal.
OpenFake	Fases 3.1–3.4 e base interna preservada nas fases seguintes	O notebook 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] seleciona por gerador/fonte, registra 1.947 imagens no teste histórico e reserva `gpt-image-2` e `nano-banana-pro` fora do treino. A 3.1 não tinha exemplos dos geradores reservados no teste registrado; a 3.2 passa a ter 155. A coleta histórica usou streaming para **selecionar e salvar** imagens; o treino posterior lê os arquivos salvos.
Community Forensics–Small	Acrescentado na 3.5 e ampliado na 3.7	Dois shards iniciais e depois dois adicionais são selecionados e persistidos. Cada seleção contém **1.500 reais e 1.500 sintéticas**; a 3.5 [experimentos/fase3_5_dados.py] registra grupos de geradores para as sintéticas, mas a fonte individual das reais aparece como `n/a`. Reais foram separados por imagem, não por fonte de captura.
AIGI	Acrescentado na 3.8	**480 pares** real/sintética, 80 pares por seis geradores, com `uid` comum. Quatro grupos entram no treino; `PA_alpha` na seleção e `SD_2.1` na calibração. Cada par fica inteiro em uma divisão. O preparo AIGI [experimentos/fase3_8_aigi_dados.py] exige imagem legível e lado curto mínimo 256, normaliza ambas as classes para JPEG.
TGIF original + SD2-fr	Acrescentado na 3.9	**768 pares** por ID da imagem original: 512 pares treino, 128 seleção, 128 calibração. São originais COCO e versões totalmente regeneradas por SD2; ambas as classes recebem JPEG RGB com lado curto 256. O preparo TGIF [experimentos/fase3_9_tgif_dados.py] não usa os arquivos `validation` ou `testing` da fonte para treinar.

As quantidades acima indicam seleções **do experimento**, não o tamanho total dos datasets públicos. Os totais combinados da 3.9 são **7.176 treino, 1.676 seleção e 1.676 calibração**, conforme o treinador [experimentos/fase3_9_tgif_finetune.py] e a saída registrada [ESTADO_PROJETO.md]. Há mais 1.947 imagens do teste OpenFake histórico conferidas por hash, sem inferência nesta versão. Em cada fase, um `manifest.json` dos dados e um `split_manifest.json` do experimento registram itens/divisões; um **hash SHA-256** permite detectar alterações exatas de conteúdo. Identidade de hash protege contra cópia binária idêntica; não detecta necessariamente uma foto recomprimida ou recortada.

4.2. Dados externos e por que seus números não são uma única série

O conjunto antigo de **20 imagens** teve 8 reais e 12 sintéticas; a avaliação da 3.4 v2 [docs/AVALIACAO_EXTERNA_FASE3_4_V2.md] registrou 11 acertos. Seu tamanho gera grande incerteza e a origem individual estava descrita como desconhecida. O benchmark **RAISE/MNW 2000 v1** reúne 1.000 fotos RAISE e 1.000 sintéticas MNW, com 25 imagens de cada um de 40 grupos sintéticos. Seis pares reais duplicados foram reparados **antes** da avaliação e o plano ficou congelado (protocolo [docs/BENCHMARK_EXTERNO_2000.md], script [experimentos/benchmark_2000.py]). As Fases 1–3.4 v2 foram reavaliadas nele com seus checkpoints congelados.

O **Mirage-Test 2000 v1** reúne 1.000 reais e 1.000 sintéticas; a seleção fixa contempla quatro domínios (`Animal`, `Human`, `Object`, `Scene`), com 250 por classe em cada domínio (protocolo [docs/BENCHMARK_EXTERNO_MIRAGE_2000.md], script de preparo [experimentos/benchmark_mirage_2000.py]). As Fases 3.5–3.9 foram comparadas nele. A primeira avaliação foi externa e congelada, porém, após resultados sucessivos se tornarem conhecidos, o mesmo Mirage **não é cego para escolhas de desenvolvimento posteriores**. As imagens não devem alterar pesos, calibração ou hiperparâmetros. Acurácias no RAISE/MNW e no Mirage não compõem uma curva única, pois os conteúdos e protocolos diferem.

4.3. Pré-processamento, aumentos e vieses

O pré-processamento padroniza a entrada, mas também pode remover sinais úteis. A Fase 1 redimensiona a imagem inteira para `224 × 224`; no treino aplica inversão horizontal, rotação moderada, mudança de brilho/contraste e compressão JPEG simulada (transformações da Fase 1 [Notebooks/fase_1/fase1_vit.ipynb]). As Fases **3.1–3.3** salvam a amostra OpenFake com lado curto limitado a 512 px em JPEG qualidade 92, depois treinam com recorte aleatório 224, inversão horizontal, desfoque leve, redução seguida de ampliação e recompressão JPEG; o teste usa cinco recortes 224 (transformações da 3.1 [Notebooks/fase_3/fase3.1_multigerador.ipynb], 3.2 [Notebooks/fase_3/fase3.2_multigerador.ipynb] e 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb]). A **3.2** aumentou a probabilidade de recompressão aleatória de 0,3 para 1,0, mantendo-a na 3.3. Nos scripts da 3.4 em diante, ela ocorre em **50%** das amostras; a 3.6 aumenta somente a chance de redução/ampliação de **0,15 para 0,50** (3.4 [experimentos/fase3_4_finetune.py], 3.6 [experimentos/fase3_6_escala.py]). Há imagens originalmente WebP no OpenFake, mas o aumento de treino conferido nesses scripts é **JPEG**, não uma simulação WebP separada.

Balanceamento importa porque um gerador com muitas imagens ou uma classe predominante poderia dominar os batches. A 3.3 e a 3.4 v1 atribuem massa igual a cada grupo dentro de cada classe; a 3.4 v2 [experimentos/amostragem_fase3_4.py] usa massa de grupo proporcional a $\sqrt{n}$, mantendo massa total de 50% por classe. Isso reduz sorteios repetidos de grupos com uma imagem, sem fazer um grupo grande pesar na proporção linear de seu tamanho. A mesma política segue nas fases recentes. A auditoria da 3.9 calculou que essa política destinou cerca de **7,835% dos sorteios a TGIF real e 1,740% a TGIF sintética**, apesar de existirem 512 exemplos de cada no treino; a causa está na normalização por **todos os grupos de cada classe**, não no tamanho bruto igual dos dois subconjuntos (auditoria interna [docs/AUDITORIA_INTERNA_FASE3_9.md]).

Por fim, **atalho de dataset** é uma pista correlacionada com o rótulo, mas irrelevante à origem que se deseja reconhecer. Um exemplo é uma fonte guardar quase todas as reais pequenas em JPEG e quase todas as sintéticas maiores em PNG. A 3.7 apresentou essa associação nos dados recém-adicionados; a 3.8 e a 3.9 padronizaram os pares novos em formato e resolução, mas não apagaram a associação dos dados históricos.

Um classificador estatístico usando apenas dimensões e tamanho do arquivo alcançou AUC **0,851 na seleção interna combinada** e **0,996 no subconjunto Community Forensics** da auditoria V3 (fechamento [docs/README_fase3_9_fechamento.md], estado [ESTADO_PROJETO.md]). Isso comprova que os metadados carregam informação sobre os rótulos **nessa amostra**, mas não prova que a rede neural dependa deles na mesma medida.

5. Funcionamento completo do treinamento

5.1. Do preparo ao primeiro batch

O **piloto** verifica se uma fonte pode ser lida, se há exemplos e se os metadados fazem sentido; não escolhe o modelo nem mede acurácia. O **preparo/congelamento** seleciona itens segundo regras fixas, baixa os arquivos escolhidos por inteiro para `/content`, normaliza as imagens, grava cópia persistente e manifesto no Drive. Nas sessões seguintes, a etapa `stage` copia os dados persistentes para o disco local temporário e confere hashes. Essa política evita que cada batch dependa de leituras remotas; a exceção histórica é a seleção inicial OpenFake por streaming (política de dados [docs/POLITICA_DADOS_FUTUROS.md], preparo TGIF [experimentos/fase3_9_tgif_dados.py]). O **smoke test** monta arquitetura, verifica partições, GPU, checkpoint, um forward finito e escrita; não ajusta pesos nem prova desempenho (modo `smoke` da 3.9 [experimentos/fase3_9_tgif_finetune.py]).

Na 3.9, o script confere a existência dos manifestos OpenFake, Community Forensics, AIGI e TGIF; verifica SHA-256 dos arquivos; exige que os dois membros de cada par AIGI/TGIF permaneçam na mesma divisão; consulta hashes do teste reservado e dos benchmarks para bloquear cópias exatas. Depois cria `DataLoader`s com o sampler e transformações. O checkpoint **inicial** é o `phase3_final.pt` da Fase 3.3, carregado com `load_state_dict(..., strict=True)`. `strict=True` rejeita chaves de pesos ausentes ou extras incompatíveis, em vez de ignorá-las. As fases 3.4–3.9 **não são uma cadeia de treino 3.4 → 3.5 → 3.6...**: cada uma documenta como ponto de partida os pesos da 3.3, o que ajuda a comparar alterações isoladas (3.9 [experimentos/fase3_9_tgif_finetune.py], 3.6 [experimentos/fase3_6_escala.py]).

5.2. Uma época de ajuste fino

Cada batch passa pela rede: pixels → ramos → gate → logits. `CrossEntropyLoss` calcula a perda, `backward()` calcula gradientes e AdamW atualiza os parâmetros. A partir da Fase 3, o ajuste fino completo deixa ViT, FFT, CNN e cabeça treináveis, mas usa taxas diferentes por grupo; na 3.9 são **5×10^{-6} ViT, 1×10^{-5} FFT e CNN, 3×10^{-5} cabeça**, com `batch_size=32`, `weight_decay=10⁻⁴` e `label_smoothing=0,05` (configuração da 3.9 [experimentos/fase3_9_tgif_finetune.py]). Taxas menores para ramos já treinados reduzem a magnitude das alterações nos pesos herdados; o efeito medido de cada taxa isolada não está disponível.

Label smoothing evita pedir probabilidade exatamente 1 para a classe rotulada: distribui pequena fração da massa para a outra classe. Isso pode reduzir excesso de confiança no treino, mas não garante calibração externa. **Weight decay** penaliza pesos grandes na atualização do AdamW e é uma forma de regularização. **Warmup** eleva gradualmente a taxa no início, quando os gradientes podem ser mais instáveis; o **decaimento cosseno** a reduz suavemente depois. O treinador reutilizado da Fase 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] implementa `LambdaLR` por época, com um warmup e curva cosseno. Não há aumento automático de épocas nas fases recentes: o limite configurado é **12** e a paciência é **3** épocas sem melhora da AUC interna.

AMP, a precisão mista, executa operações apropriadas em `float16` na GPU para reduzir custo, enquanto `GradScaler` evita que gradientes pequenos se percam. Antes de limitar o gradiente, o código desfaz a escala; `clip_grad_norm_` limita a norma a **1,0** para evitar atualizações excepcionalmente grandes. Essas técnicas

aparecem no laço de treino da 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] reutilizado pelo script da 3.9 [experimentos/fase3_9_tgif_finetune.py]. A existência delas é verificada; não há no repositório um experimento controlado que quantifique separadamente quanto cada uma melhorou a acurácia.

5.3. Seleção, calibração e arquivos salvos

Ao fim de cada época, a seleção interna usa **recorte central** para calcular perda, AUC e outras métricas. O `best.pt` guarda a época de maior AUC. Se não houver melhora durante três épocas consecutivas, o **early stopping** encerra antes do máximo; ele evita continuar atualizando um modelo sem ganho nesse critério, mas não garante o melhor desempenho externo. O `last.pt` guarda pesos correntes, otimizador, scheduler, scaler, época e histórico para retomada; `history.csv` mostra a trajetória; `best.pt` preserva a melhor época (treinador 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb], script 3.9 [experimentos/fase3_9_tgif_finetune.py]).

Depois de carregar `best.pt`, o código usa **somente a partição de calibração** com cinco recortes para ajustar a temperatura `T` por minimização da entropia cruzada e escolher o limiar que maximiza a acurácia balanceada nessa partição. Temperatura e limiar são salvos no checkpoint final `phase3_final.pt`, juntamente com os pesos. `internal_metrics.json` registra a AUC de seleção e a acurácia balanceada **aparente** da calibração; esta última é medida nos mesmos exemplos usados para escolher o limiar e, portanto, não é estimativa imparcial de teste. `training_complete.json` assinala conclusão do treino interno, não conclusão do teste externo. As operações estão no script da 3.9 [experimentos/fase3_9_tgif_finetune.py].

Na Fase 2, a estratégia era diferente em um ponto: a seleção do melhor checkpoint usava **F1**, em duas etapas (primeiro com os ramos herdados congelados, depois ajuste conjunto), e a calibração ficava em uma segunda metade da validação (notebook da Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb]). Na Fase 1, o melhor checkpoint era escolhido por F1 em uma validação própria, e a temperatura era ajustada na mesma validação; o teste externo RAISE/MNW posterior usou o checkpoint `phase1_best_model.pt` sem temperatura persistida, logo adotou `T=1` e limiar `0,5` (notebook da Fase 1 [Notebooks/fase_1/fase1_vit.ipynb], avaliador histórico [experimentos/avaliar_benchmark_2000.py]).

5.4. Retomada e integridade

O modo `auto` das fases recentes verifica `training_complete.json`; se existir, não treina outra vez. Se há `last.pt`, transforma a ação em `resume`; caso contrário, inicia uma execução nova **apenas no diretório da versão**. Uma retomada compara a configuração e o manifesto de partições salvos, recusando diferenças. O notebook de execução prepara o runtime, mas não inicia o treino ao abrir. Pastas separadas de modelos e resultados preservam versões anteriores (treinador da 3.9 [experimentos/fase3_9_tgif_finetune.py], notebook Colab da 3.9 [Notebooks/fase_3/rodar_fase3_9_tgif_colab.ipynb]). Essa proteção reduz sobrescritas e troca silenciosa de dados. Ela não faz uma desconexão desaparecer: a cópia temporária de `/content` precisa ser refeita após reiniciar o Colab.

6. Evolução histórica das versões

6.1. Visão comparativa

Versão implementada	Arquitetura e ponto de partida	Mudança documentada	Seleção/medição principal
1	ViT-Tiny + FFT-CNN, fusão por concatenação, treinados inicialmente	Detector binário com faces e COCO real	F1 na validação própria; teste próprio

Versão implementada	Arquitetura e ponto de partida	Mudança documentada	Seleção/medição principal
2	Acrescenta EfficientNet-B0 pré-treinado e gate; ViT/FFT herdados da 1	Duas etapas de treino e calibração separada	F1 na validação própria; teste próprio
3.1	Mesma arquitetura da 2, inicializada pela 2	Ajuste fino em OpenFake multigerador, recortes e augmentations	AUC interna; teste OpenFake
3.2	Mesma arquitetura	Holdout explícito de `gpt-image-2` / `nano-banana-pro`, recompressão JPEG aleatória em todas as amostras de treino e mais épocas configuradas	AUC interna; teste OpenFake
3.3	Mesma arquitetura	Nova execução e auditorias de compressão/dados; o código das transformações já existia na 3.2	AUC interna; teste OpenFake
3.4 v1	Mesma arquitetura; pesos iniciais da 3.3	Ajuste fino com taxa menor, 12 épocas máximas e JPEG aleatório em metade dos batches	AUC interna; teste OpenFake
3.4 v2	Mesmo checkpoint inicial e manifesto da v1	Amostrador `√n` por grupo; sem replay efêmero; partições registradas	AUC interna; teste OpenFake
3.5	Mesma arquitetura e pesos iniciais da 3.3	Acrescenta seleção interna Community Forensics–Small	AUC interna; Mirage comparativo
3.6	Mesmos dados/partições da 3.5	`RandomDownUp` com probabilidade 0,50 em vez de 0,15	AUC interna; Mirage comparativo
3.7	Mesma arquitetura e pesos iniciais da 3.3	Expande Community Forensics–Small com nova seleção	AUC interna; Mirage comparativo
3.8	Mesma arquitetura e pesos iniciais da 3.3	Acrescenta 480 pares AIGI com formato/escala pareados	AUC interna; Mirage comparativo
3.9	Mesma arquitetura e pesos iniciais da 3.3	Acrescenta 768 pares TGIF original/SD2-fr	AUC interna; Mirage comparativo

Os arquivos principais correspondentes são os notebooks das Fases 1–3.3 [Notebooks/], os scripts de 3.4 v1 [experimentos/fase3_4_finetune.py], 3.4 v2 [experimentos/fase3_4_finetune_v2.py], 3.5 [experimentos/fase3_5_finetune.py], 3.6 [experimentos/fase3_6_escala.py], 3.7 [experimentos/fase3_7_finetune.py], 3.8 [experimentos/fase3_8_aigi_finetune.py] e 3.9 [experimentos/fase3_9_tgif_finetune.py]. O histórico Git local registra a criação das variantes e correções de coleta/retomada; ele mostra **quando** uma correção entrou no código, enquanto as métricas dependem das saídas executadas e do conjunto avaliado. Por exemplo, os commits de reparo AIGI e TGIF alteraram a seleção/leitura de arquivos, não uma arquitetura nova.

6.2. Fases 1 e 2: fundação e transferência

Fase 1. O notebook [Notebooks/fase_1/fase1_vit.ipynb] implementou ViT-Tiny e FFT-CNN, concatenou os dois vetores e treinou com entropia cruzada, AdamW, warmup/cosseno, label smoothing 0,1, AMP e clipping. O código comenta que uma rodada anterior ficou em 74,5%, mas não há neste guia um protocolo verificável dessa rodada; o resultado preservado do **teste próprio de faces**, após treino com faces e COCO real adicional, é **88,14% de acurácia**, precisão **83,96%**, recall **94,31%** e F1 **88,83%** (saída do notebook [Notebooks/fase_1/fase1_vit.ipynb]). A saída alta nesse teste próprio não é evidência de 85% em fotos externas variadas. O notebook mostra uma temperatura calibrada de aproximadamente 1,07, mas o arquivo usado no benchmark externo foi o checkpoint de melhores pesos, sem essa calibração persistida.

Fase 2. O notebook [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb] adicionou o ramo EfficientNet-B0 pré-treinado em ImageNet e o gate. Recarregou estritamente os ramos ViT/FFT da Fase 1; treinou primeiro componentes novos com os ramos herdados congelados e depois liberou o conjunto. Configurou até 5 épocas na primeira etapa e 18 na segunda, seleção por F1, AdamW, AMP e calibração separada. O **teste próprio** registrou **99,74% de acurácia**, precisão **99,54%**, recall **99,94%**, F1 **99,74%** e especificidade **99,54%**, com temperatura **0,3909** e limiar **0,725** no checkpoint (métricas locais [modelos/fase2_gate_vit_fft_cnn/final_metrics.json]). Esses resultados mostram ajuste ao domínio testado; a AUC da Fase 2 no teste OpenFake posteriormente usada como referência da 3.3 foi cerca de **0,5617**, com detecção de sintéticas quase nula no limiar original (saída da 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb]).

6.3. Fases 3.1 a 3.3: generalização no OpenFake

3.1 introduziu ajuste fino completo da arquitetura de três ramos em imagens OpenFake de vários geradores/fonte, amostragem por grupo e avaliação de cinco recortes. O notebook [Notebooks/fase_3/fase3.1_multigerador.ipynb] configurou oito épocas, registrou 5.187 treino, 619 seleção, 619 calibração e 1.947 teste, mas informou **zero imagens** dos geradores holdout no teste dessa execução; portanto não é avaliação de holdout desses geradores. A AUC OpenFake foi **0,7832** e a acurácia no limiar interno **69,85%** (métricas [modelos/fase3_multigerador/final_metrics.json]).

3.2 mudou o protocolo de reserva de geradores: o notebook [Notebooks/fase_3/fase3.2_multigerador.ipynb] confirmou `gpt-image-2` e `nano-banana-pro` no teste e ausentes do treino, com **155 imagens** desses grupos no teste. Também elevou de **0,3 para 1,0** a probabilidade de recompressão JPEG aleatória durante o treino, configurou até 20 épocas e registrou 5.642 treino, 679 seleção, 679 calibração. No OpenFake, AUC **0,7993** e acurácia **71,24%** (métricas [modelos/fase3_holdout_v2/final_metrics.json]). Como mais de um aspecto do conjunto/protocolo difere da 3.1, a diferença de métricas não pode ser atribuída só ao número de épocas, ao holdout ou à recompressão.

3.3 deu foco documental e experimental à compressão e à auditoria dos dados: a configuração do notebook [Notebooks/fase_3/fase3.3_compressao.ipynb] limita o lado curto salvo a 512, grava JPEG qualidade 92 e aplica recompressão JPEG durante o treino, além dos recortes. **A comparação direta das células de transformação e de treino mostra que esse código já estava na 3.2**; portanto, não é correto afirmar que a 3.3 inventou a recompressão ou mudou a arquitetura em relação à 3.2. Ela registrou uma nova execução e acrescentou células de exploração/auditoria, com **6.429 imagens de treino, 777 de seleção, 777 de calibração e 1.947 de teste**; o replay de faces podia entrar se a pasta efêmera existisse. AUC OpenFake **0,8041**, acurácia **71,14%**, recall sintético **57,13%** e especificidade real **84,4%** (métricas [modelos/fase3_3_compressao_v1/final_metrics.json]). O projeto não tem ablação que isole o impacto da compressão do de mudanças de amostra, replay ou demais configurações. A 3.3 virou o **checkpoint inicial comum** dos ajustes finos posteriores.

6.4. Fase 3.4: duas amostragens sobre a mesma base

A **3.4 v1** usa o checkpoint 3.3, até 12 épocas e taxas menores; sua coleta fixa **2.032 imagens no manifesto de treino OpenFake** e **1.947 no teste** (script [experimentos/fase3_4_finetune.py], guia [docs/README_fase3_4.md]). As partições de treino/seleção/calibração são derivadas desse manifesto; o replay opcional dependia de `/content/data/faces`, cuja seleção não foi persistida. O melhor checkpoint da execução foi o da época 1, e o histórico parou na época 4. No teste OpenFake, acurácia **71,55%**, AUC **0,8165**, precisão **77,87%**, recall **57,97%**, F1 **66,46%**, especificidade **84,4%** (comparativo [docs/COMPARATIVO_RESULTADOS.md]).

A **3.4 v2** manteve arquitetura, checkpoint inicial, dados preparados e configuração de treino, mas mudou a massa dos grupos para `vn` e desativou replay efêmero, registrando divisões em `split_manifest.json` (script [experimentos/fase3_4_finetune_v2.py], guia [docs/README_fase3_4_v2.md]). Registrou **1.668 treino, 182 seleção, 182 calibração**; a melhor época foi a 6, com AUC de seleção **0,9671**, e o treino encerrou após a época 9. No mesmo teste OpenFake, alcançou **72,88% de acurácia**, AUC **0,8222**, precisão **78,05%**, recall **61,56%**, F1 **68,83%**, especificidade **83,6%**. A v2 acertou 34 sintéticas a mais que a v1, mas produziu oito FP adicionais. A

comparação **não isola perfeitamente só o sampler** se o replay opcional da v1 esteve ativo; é correto dizer que o resultado observado foi melhor nesse teste, não provar uma causa única. Na avaliação antiga de 20 imagens, a v2 acertou **11/20 (55%)**, com 9 FN (relatório externo [docs/AVALIACAO_EXTERNA_FASE3_4_V2.md]).

6.5. Fases 3.5 a 3.9: dados, robustez e pares

3.5 incorporou uma seleção interna Community Forensics–Small à base OpenFake, preservando a arquitetura e o ponto de partida 3.3. O piloto conferiu os shards; o preparo fixou 3.000 novas imagens, com real_source `n/a` para as fotos reais. Foram **3.594/719/719** imagens nas divisões de treino/seleção/calibração. A melhor época foi a 10/12, AUC interna de seleção **0,9488**, `T=0,44095` e limiar `0,445`. O diagnóstico interno com cinco recortes, na seleção **já usada para escolher o modelo**, encontrou **90,68% de acurácia**; no Mirage, a acurácia foi **60,45%** (script [experimentos/fase3_5_finetune.py], guia [docs/README_fase3_5.md]). Essa diferença demonstra que as duas avaliações respondem a perguntas distintas.

3.6 testou uma alteração mais controlada: manteve dados, divisões e checkpoint inicial da 3.5 e elevou `RandomDownUp` de 0,15 para 0,50 (script [experimentos/fase3_6_escala.py], guia [docs/README_fase3_6.md]). A melhor época foi a 12, AUC interna **0,9472**, e a avaliação Mirage deu **57,2%**, abaixo dos 60,45% da 3.5. O resultado não sustentou ganho com essa intensidade de aumento nas condições testadas; uma única execução por variante não explica toda a variabilidade possível.

3.7 acrescentou outra seleção interna Community Forensics–Small, sem trocar arquitetura ou checkpoint inicial; as divisões cresceram para **5.512/1.260/1.260** (scripts de dados [experimentos/fase3_7_dados.py] e treino [experimentos/fase3_7_finetune.py]). A melhor época foi a 11, AUC interna **0,96995**. No Mirage, acurácia **62,3%**, AUC **0,69083**: é o maior valor observado das fases 3.5–3.9 nesse benchmark, mas continua distante de 85%.

3.8 acrescentou **480 pares AIGI** mantendo os componentes e o checkpoint inicial, com **6.152/1.420/1.420** imagens. O preparo precisou ampliar a busca de shards quando alguns URLs reais ou imagens sintéticas não satisfizeram os critérios; as correções preservaram o plano e verificaram os bytes antes do congelamento (preparo [experimentos/fase3_8_aigi_dados.py], guia [docs/README_fase3_8_execucao.md]). A melhor época foi a 12, AUC interna **0,96031**, `T=0,52290`, limiar `0,31`. No Mirage, houve **59,9% de acurácia**, AUC **0,64127**, recall sintético **55,7%** e especificidade real **64,1%**: mais sintéticas detectadas que na 3.7, mas mais fotos reais acusadas indevidamente. A melhora de recall isolado não compensou a perda de acertos totais.

3.9 acrescentou **768 pares TGIF original/SD2-fr** aos dados anteriores, com **7.176/1.676/1.676** imagens (preparo [experimentos/fase3_9_tgif_dados.py], treino [experimentos/fase3_9_tgif_finetune.py]). O preparo leu membros dos arquivos `training` pelo **caminho completo de categoria e nome**, porque um mesmo ID pode aparecer com recortes distintos; o ID inteiro permaneceu numa só divisão (registro técnico [docs/README_fase3_9_execucao.md]). O treino executou 12 épocas; a melhor foi a 10, AUC interna de seleção **0,89882**, `T=0,91879`, limiar `0,47`. No Mirage, acurácia **58,75%**, AUC **0,63281**, recall sintético **43,1%**, especificidade real **74,4%** (resultado registrado [ESTADO_PROJETO.md]). A auditoria interna posterior contou **59 FP e 182 FN em 1.676 imagens de seleção**; **115 dos 128 TGIF sintéticos** foram FN. Isso aponta uma dificuldade concentrada nesse subconjunto, sem provar que a adição dos pares seja a única causa da métrica externa. O resumo da seleção interna deve ser lido com a ressalva de que ela escolheu o checkpoint e de que a revisão visual encontrou imagens quase repetidas envolvendo a calibração (auditoria [docs/AUDITORIA_INTERNA_FASE3_9.md], revisão visual [docs/REVISAO_VISUAL_FASE3_9.md]).

7. Avaliação e interpretação dos resultados

7.1. Primeiro, a matriz de confusão

Como **sintética é a classe positiva**, os quatro casos são: `TP`, sintética corretamente detectada; `FN`, sintética classificada como real; `TN`, foto real corretamente preservada como real; `FP`, foto real acusada de sintética. Para avaliar o detector usado por pessoas, os dois erros importam: FN pode deixar passar uma imagem artificial; FP pode desacreditar uma foto genuína. A função de métricas da Fase 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] e os avaliadores Mirage [experimentos/avaliar_fase3_9_mirage.py] seguem essa convenção.

Imagine um teste didático com 100 fotos reais e 100 sintéticas. Se a rede acertar 80 reais e 70 sintéticas, então `TN=80`, `FP=20`, `FN=30`, `TP=70`. São **150/200 = 75% de acurácia**. O exemplo mostra por que dizer apenas "acertou 75%" oculta quais erros aconteceram. A matriz de confusão deve acompanhar a acurácia, principalmente se o custo dos dois erros for diferente. O exemplo não é resultado do TCC.

Métrica	Fórmula, com sintética positiva	Pergunta respondida no exemplo
Acurácia	`(TP + TN)/(TP + TN + FP + FN)`	75% de todas as imagens foram classificadas corretamente.
Precisão (*precision*)	`TP/(TP + FP)`	Das chamadas "sintéticas", `70/90 ≈ 77,8%` eram sintéticas.
Recall ou sensibilidade sintética	`TP/(TP + FN)`	Das sintéticas existentes, `70/100 = 70%` foram encontradas.
Especificidade real ou TNR	`TN/(TN + FP)`	Das reais existentes, `80/100 = 80%` foram preservadas.
F1	`2 × precisão × recall/(precisão + recall)`	Combina precisão e recall; aqui cerca de 73,7%.
Acurácia balanceada	`(recall + especificidade)/2`	Dá peso igual ao acerto das duas classes; aqui 75%.

Precisão não é acurácia. No Mirage da 3.9, o grupo observou 58,75% de acurácia e 43,1% de recall sintético; uma precisão eventualmente alta entre os alarmes de "sintética" não significaria que 85% de todas as imagens foram acertadas. Também é possível obter 50% de acurácia num conjunto equilibrado simplesmente chamando todas as imagens de reais: `TN=1.000`, `FN=1.000`, `TP=0`. Essa foi aproximadamente a conduta do limiar histórico da Fase 2 no RAISE/MNW: acurácia 50% e recall zero, embora sua AUC fosse 0,6426.

7.2. AUC, probabilidade, temperatura e limiar

A **curva ROC** mede, à medida que se varia o limiar, a relação entre taxa de sintéticas detectadas (`TPR = recall`) e taxa de fotos reais acusadas (`FPR = FP/(FP+TN)`). A **AUC ROC** é a área sob essa curva. Ela avalia a capacidade de **ordenar** sintéticas acima de reais em vários pontos de corte; 0,5 corresponde a ordenação aleatória em expectativa, 1 a separação perfeita naquele conjunto. Outra interpretação: aproximadamente a probabilidade de uma sintética sorteada receber pontuação maior que uma real sorteada, com tratamento adequado para empates. Não se obtém AUC somente da matriz de confusão de um limiar: são necessárias as pontuações por imagem. Uma AUC razoável pode coexistir com acurácia ruim quando o limiar operacional não se transfere ao novo domínio.

O modelo devolve dois **logits** `z_real` e `z_sintética`, que são escores livres, não porcentagens. `softmax(z/T)` transforma-os em dois números positivos que somam 1; o componente da classe 1 é `p_sintética` e o da classe 0 é `p_real`. A temperatura `T>0` é ajustada **na calibração interna**: com os mesmos logits, `T<1` torna a distribuição mais extrema, `T>1` a suaviza. A temperatura **não altera a ordem das classes nem a AUC** quando é um único escalar positivo aplicado a todos os logits; ela altera as probabilidades e pode alterar as decisões caso se mantenha um limiar numérico fixo fora de 0,5. Seu valor "bom" não é o mais próximo de 0 ou 1: deve ser julgado pela calibração e pelo desempenho no conjunto apropriado. Por exemplo, `T≈0,44` na 3.5 e `T≈0,92` na 3.9 são ajustes de escala de saídas diferentes, não notas de qualidade comparáveis.

O **limiar** é outra operação: se `p_sintética ≥ limiar`, o rótulo operacional é "sintética"; caso contrário, "real". Aumentá-lo costuma reduzir falsos positivos, mas também pode deixar passar mais sintéticas. No projeto, ele é escolhido sobre a **calibração interna** para maximizar acurácia balanceada, após a temperatura (funções `fit_temperature` e `pick_threshold` da 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb], reaproveitadas na 3.9 [experimentos/fase3_9_tgif_finetune.py]). Logo, `0,47` não é "melhor" nem "pior" que `0,31` por ser mais próximo de 0,5 ou de 1. A diferença só ganha sentido junto às distribuições de pontuação e aos erros medidos. A relação entre ambas é **conjunta**: ao comparar fases, não se deve atribuir uma mudança de recall apenas ao limiar se os pesos também mudaram.

Calibração significa que, numa população de muitas imagens às quais o sistema atribui probabilidade próxima de 70%, aproximadamente 70% deveriam pertencer à classe indicada. Ajustar `T` minimizando perda logarítmica em dados internos é uma tentativa de melhorar essa correspondência; **não comprova calibração para toda imagem de usuário**. O `ECE` (*expected calibration error*) registrado na 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb] resume a diferença entre confiança média e frequência de acerto em intervalos de confiança, mas depende do conjunto e dos intervalos escolhidos. Assim, a frase "70% de chance estimada" descreve a saída do modelo, não uma prova de origem.

7.3. Três contextos de resultado que não devem ser confundidos

Contexto	O que foi efetivamente observado	Limite de interpretação
Testes próprios de faces da Fase 1 e da Fase 2	88,14% e 99,74% de acurácia, respectivamente (notebooks [Notebooks/]). COCO real foi adicionado ao treino, não a esses testes.	Domínio próprio; não estima desempenho em geradores e fotos externos variados.
Teste OpenFake de 1.947 imagens	3.4 v1: 71,55%; 3.4 v2: 72,88% (comparativo [docs/COMPARATIVO_RESULTADOS.md]).	Teste distinto do benchmark externo; o OpenFake reservado das fases 3.5–3.9 **não** foi inferido nessas execuções.
20 imagens antigas	3.4 v2: 11/20 = 55%, com 9 FN (relatório [docs/AVALIACAO_EXTERNA_FASE3_4_V2.md]).	Amostra pequena, com origem individual incompleta; cada caso muda a acurácia em 5 pontos percentuais.
RAISE/MNW 2000	Fases 1–3.4 v2, 1.000 por classe (notebook de avaliação [Notebooks/fase_3/avaliar_fases_benchmark_2000_colab.ipynb]).	Avaliação congelada desses checkpoints, mas já observada depois; não se mistura às porcentagens Mirage.
Mirage-Test 2000	Fases 3.5–3.9, 1.000 por classe (protocolos [docs/BENCHMARK_EXTERNO_MIRAGE_2000.md]).	Comparação nas mesmas imagens, porém o conjunto foi visto ao longo do desenvolvimento; não é validação cega de escolhas posteriores.

RAISE/MNW, avaliação reexecutada dos checkpoints históricos:

Modelo	Acurácia	AUC	Recall sintético	Especificidade real
Fase 1	50,20%	0,5425	0,6%	99,8%

Modelo	Acurácia	AUC	Recall sintético	Especificidade real
Fase 2	50,00%	0,6426	0,0%	100,0%
Fase 3.1	55,05%	0,5568	31,7%	78,4%
Fase 3.2	56,55%	0,6048	28,8%	84,3%
Fase 3.3	59,95%	0,7384	22,6%	97,3%
Fase 3.4 v1	59,20%	0,7362	20,7%	97,7%
Fase 3.4 v2	**62,10%**	0,7375	28,9%	95,3%

Fonte: saídas gravadas no notebook RAISE/MNW [Notebooks/fase_3/avaliar_fases_benchmark_2000_colab.ipynb] e protocolo [docs/BENCHMARK_EXTERNO_2000.md]. A Fase 1 usou `T=1` e limiar 0,5 porque o checkpoint avaliado não continha calibração; a Fase 2 usou seus valores persistidos. O protocolo de entrada também difere: as Fases 1–2 usam redimensionamento integral, e a Fase 3, cinco recortes de 224. Assim, a tabela mede sistemas completos nos respectivos protocolos, não só pesos sob uma entrada idêntica.

Mirage-Test 2000 v1, comparação dos checkpoints posteriores:

Modelo	Acurácia	AUC	Recall sintético	Especificidade real	Matriz `TN/FP/FN/TP` quando registrada
3.5	60,45%	0,6800	35,1%	85,8%	858 / 142 / 649 / 351
3.6	57,20%	0,6304	28,8%	85,6%	856 / 144 / 712 / 288, derivada de 1.000 por classe
3.7	**62,30%**	**0,6908**	44,0%	80,6%	806 / 194 / 560 / 440, derivada de 1.000 por classe
3.8	59,90%	0,6413	55,7%	64,1%	641 / 359 / 443 / 557
3.9	58,75%	0,6328	43,1%	74,4%	744 / 256 / 569 / 431

Fonte: estado do projeto [ESTADO_PROJETO.md], guia da 3.6 [docs/README_fase3_6.md], guia da 3.8 [docs/README_fase3_8_execucao.md], execução da 3.9 [docs/README_fase3_9_execucao.md] e saídas dos notebooks de avaliação correspondentes em Notebooks/fase_3 [Notebooks/fase_3/]. As matrizes 3.6 e 3.7 marcadas como derivadas decorrem aritmeticamente das 1.000 imagens por classe e dos recalls/especificidades informados; não são novas inferências. A diferença 3.7 → 3.8 mostra por que perseguir apenas recall não basta: a 3.8 detectou mais sintéticas, mas acusou 359 fotos reais em vez de 194 e perdeu acurácia total. A 3.9 acertou **1.175/2.000** imagens; a 3.8 acertara 1.198. A comparação pareada registrou 135 acertos exclusivos da 3.8, 112 exclusivos da 3.9, 1.063 acertos comuns e 690 erros comuns (resultado da 3.9 [ESTADO_PROJETO.md]).

O Mirage da 3.9 tem acurácia 58,75%, não 85%. A **seleção interna** da mesma versão tem `TN=699`, `FP=59`, `FN=182`, `TP=736`: `1.435/1.676 ≈ 85,62%` (auditoria [docs/AUDITORIA_INTERNA_FASE3_9.md]). Essa seleção participou da escolha do checkpoint; portanto, os 85,62% não satisfazem a meta de acurácia externa. A própria diferença entre esses contextos é um resultado científico relevante: o modelo aprendeu regularidades úteis internamente, mas não demonstrou a transferência desejada no benchmark observado.

7.4. Incerteza e leitura científica

Um resultado medido em `N` imagens é uma estimativa, não uma propriedade fixa do modelo em todas as imagens. O avaliador Mirage da 3.9 [experimentos/avaliar_fase3_9_mirage.py] registra um **intervalo de Wilson de 95%** para a acurácia. Um intervalo desse tipo quantifica parte da variação amostral sob hipóteses estatísticas usuais; ele **não incorpora automaticamente** viés de seleção, dependência entre imagens ou mudança de domínio. Com 20 imagens, a incerteza é muito maior que com 2.000; com 2.000 imagens correlacionadas por gerador/conteúdo, o número nominal de casos também pode superestimar a diversidade efetiva.

O protocolo de avaliação mantém manifesto, hash do checkpoint, hash do script, temperatura e limiar registrados antes da inferência (bloqueio da 3.9 [experimentos/avaliar_fase3_9_mirage.py]). Isso torna a comparação auditável e impede substituir o resultado silenciosamente. Mesmo assim, **ausência de duplicatas SHA-256 exatas não prova independência visual**. A revisão de 19 candidatos internos da 3.9 [docs/REVISAO_VISUAL_FASE3_9.md] encontrou duas fotografias quase idênticas envolvendo calibração, e um par sintético da mesma cena com conteúdo alterado. Isso limita a leitura da calibração interna; o efeito numérico sobre as métricas não foi medido. Não se deve concluir, por esse achado isolado, que a AUC de seleção tenha sido inflada em uma quantidade específica.

8. Funcionamento do sistema implementado

8.1. Arquitetura, pesos e checkpoint são coisas diferentes

Arquitetura é a definição das camadas e de como elas se conectam. **Pesos** são os valores numéricos aprendidos nessas camadas. Um **checkpoint** é um arquivo que guarda pesos e, conforme seu propósito, configuração, época, estados do otimizador e calibração. Na 3.9, `last.pt` serve à retomada; `best.pt` guarda a melhor época por AUC interna; `phase3_final.pt` guarda o modelo escolhido com temperatura e limiar para inferência (script [experimentos/fase3_9_tgif_finetune.py]). Instanciar `Phase2Detector` monta a arquitetura; chamar `load_state_dict` carrega os pesos. A frase antiga “Fase 2 carregada” podia ser mal interpretada quando o script logo depois carregava o checkpoint da 3.3; os scripts recentes distinguem as duas ações.

O carregamento de um checkpoint precisa usar a classe de arquitetura compatível. Os avaliadores históricos [experimentos/avaliar_benchmark_2000.py] reconstroem `Phase1Detector` ou `Phase2Detector` das células dos notebooks, verificam configuração e carregam `model_state_dict` com `strict=True`. O avaliador Mirage da 3.9 [experimentos/avaliar_fase3_9_mirage.py] confere hashes de arquitetura e checkpoint antes de produzir predições. Os pesos são arquivos grandes no Drive e não ficam no Git; o código versionado no GitHub é sincronizado para um snapshot `/content/tcc_codigo/<sha>`. O notebook Colab da 3.9 [Notebooks/fase_3/rodar_fase3_9_tgif_colab.ipynb] prepara essa sessão; os scripts Python mantêm a lógica principal.

8.2. Uma imagem do início ao fim

Suponha uma imagem JPEG enviada para classificação. O avaliador a abre com PIL e converte para RGB. No **protocolo Mirage das fases recentes**, se o lado curto for menor que 224 px, há ampliação bicúbica; depois são extraídos **cinco recortes de 224 px**: quatro cantos e centro. Cada recorte vira tensor e recebe a normalização do projeto. O avaliador passa os cinco recortes pela rede e faz a **média dos dois logits**, recorte por recorte, antes da temperatura (protocolo da 3.9 [experimentos/avaliar_fase3_9_mirage.py], avaliador base [experimentos/benchmark_mirage_2000.py]). Fazer média das **probabilidades** seria outro algoritmo e não reproduziria esse resultado.

Imagine que a média dos logits seja `[1,2; 0,8]` para real/sintética e que o checkpoint tenha `T=0,9188`. O sistema calcula `softmax([1,2; 0,8]/T)`; isso produz `p_sintética` próxima de 0,393 e `p_real` próxima de 0,607. Com limiar congelado 0,47, a **classificação operacional seria “real”**. Os números são um exemplo

didático; o código não fornece esses logits para uma foto específica. Um valor como 39,3% não comprova a origem. Se a fotografia estivesse fora da distribuição usada na calibração, a probabilidade poderia estar mal calibrada mesmo com a operação matemática correta.

As **frases pré-definidas** estão implementadas em `descrever_predicao` [experimentos/linguagem_predicao.py]. A função recebe `p_sintética` e o limiar, devolve também `p_real`, classe operacional e uma mensagem. As faixas de linguagem são `<20%`, `20–<40%`, `40–<60%`, `60–<80%` e `≥80%` de probabilidade sintética. Elas mudam **apenas a maneira de comunicar** o resultado; não são novos limiares de treino. Um caso com `p_sintética=40%` e limiar 31% pode receber texto de "chances próximas", mas classe operacional "sintética"; mostrar ambos evita incoerência. A existência da função compartilhada não significa que todos os notebooks históricos já tenham sido reescritos para chamar essa função; os benchmarks preservados continuam principalmente em CSVs numéricos.

O repositório também contém a inferência exploratória `predict_pils` no notebook 3.3 [Notebooks/fase_3/fase3.3_compressao.ipynb], que recebe imagens PIL, aplica seu pré-processamento histórico e devolve probabilidades e pesos médios do gate. Os pesos do gate mostram a mistura aprendida, **não a razão causal completa** da decisão. Não há, neste repositório, endpoint HTTP ou interface web de produção que receba imagens de usuários em funcionamento. Uma demonstração citada na documentação é tratada como artefato separado; seu código não foi usado para inferir funcionalidades desta base.

8.3. Arquivos, bibliotecas e rastreabilidade

`torch` e `torchvision` implementam redes, transformações, DataLoaders e treinamento; PIL abre e converte imagens; NumPy auxilia cálculos; `scikit-learn` calcula AUC e outras métricas; notebooks JSON organizam a execução no Colab. O código remoto é versionado em Git; dados selecionados, manifestos, checkpoints e resultados ficam sob `/content/drive/MyDrive/TCC_Final` (README [README.md]). O espelhamento de código para o Drive [experimentos/espelhar_codigo_drive.py] é uma cópia de `.py` / `.ipynb`; o snapshot por commit usado na execução é o controle de versão efetivo. Um manifesto e seu SHA-256 respondem "quais imagens foram usadas?"; `experiment_config.json` responde "com quais parâmetros?"; `history.csv` responde "como evoluiu a seleção?"; `phase3_final.pt` responde "quais pesos e calibração foram aplicados?"; `metrics.json` e `predictions.csv` respondem "o que ocorreu no conjunto avaliado?".

9. Decisões técnicas, dificuldades e soluções

9.1. Por que combinar três ramos

O problema exige comparar informação local, relações entre regiões e padrões de frequência. A Fase 1 já combinava ViT e FFT; a Fase 2 acrescentou uma CNN pré-treinada e uma fusão aprendida para aproveitar características espaciais hierárquicas (arquitetura da Fase 2 [Notebooks/fase_2/fase2_gate_vit_fft_cnn_corrigido.ipynb]). A alternativa de concatenação aparece no código da Fase 2, mas o caminho principal usa gate. Sua vantagem potencial é adaptar o peso dos ramos por imagem; seu compromisso é maior complexidade e risco de aprender atalhos também nos pesos de fusão. O salto no **teste próprio** da Fase 2 é observável, mas ocorreu junto com novos pesos, treino em duas etapas e calibração; não permite atribuir todo o ganho ao EfficientNet ou ao gate isoladamente.

9.2. Por que usar ajuste fino e não reiniciar a cada fase

Treinar uma arquitetura de cerca de **10,06 milhões de parâmetros** do zero em cada pequeno conjunto do Colab seria custoso e exigiria mais dados. A Fase 2 aproveitou pesos ViT/FFT da 1 e CNN ImageNet; as fases 3.4–3.9 carregam estritamente o checkpoint da 3.3 (3.9 [experimentos/fase3_9_tgif_finetune.py]). Isso preserva

características já aprendidas e permite testar alterações incrementais de dados/augmentation. O compromisso é herdar também limitações do modelo inicial. A AUC externa das fases posteriores não cresceu monotonicamente, mostrando que mais dados ou mais épocas não garantem transferência.

9.3. Por que controlar grupos, formatos e pares

A coleta OpenFake por gerador/fonte evita que uma única origem domine a seleção e permite reservar geradores no teste. A v2 da 3.4 reduziu a repetição excessiva de grupos minúsculos com massa `vn` (sampler [experimentos/amostragem_fase3_4.py]). Community Forensics acrescentou muitos geradores, mas as reais vieram com fonte individual `n/a` e mostraram associação entre classe e resolução (guia da 3.5 [docs/README_fase3_5.md], auditoria da 3.9 [docs/AUDITORIA_INTERNA_FASE3_9.md]). Os pares AIGI e TGIF dão real e sintética relacionadas pelo mesmo UID/ID, ambas processadas em JPEG e 256 px. Isso reduz diferenças triviais dentro **desses pares**, sem eliminar todos os vieses históricos nem garantir equivalência de conteúdo em todo o treino.

A coleta também teve problemas concretos: fontes AIGI reais fora do ar ou com imagem abaixo do mínimo, e membros TGIF com mesmo basename em categorias distintas. O código AIGI passou a validar os bytes antes de congelar a seleção; o TGIF passou a buscar membro pelo caminho completo da categoria. Essas correções de **integridade dos dados** são visíveis em fase3_8_aigi_dados.py [experimentos/fase3_8_aigi_dados.py], fase3_9_tgif_dados.py [experimentos/fase3_9_tgif_dados.py] e no histórico Git. Elas não representam, sozinhas, uma melhoria neural medida.

9.4. Por que separar seleção, calibração e teste

Escolher uma época, temperatura e limiar no mesmo teste externo daria ao modelo informação indireta sobre esse teste. O fluxo recente usa AUC na seleção interna e calibração em uma divisão separada; só depois congela esses valores para a avaliação Mirage (treinador 3.9 [experimentos/fase3_9_tgif_finetune.py], avaliador 3.9 [experimentos/avaliar_fase3_9_mirage.py]). Isso melhora a disciplina metodológica. Entretanto, a revisão visual interna [docs/REVISAO_VISUAL_FASE3_9.md] encontrou imagens praticamente repetidas entre seleção/calibração e treino/calibração. Portanto, a separação por caminho e hash exato não garantiu separação completa de conteúdo visual; a temperatura e o limiar da 3.9 devem ser lidos com essa ressalva. Não há quantificação do impacto dessa repetição nos indicadores.

9.5. Por que usar Drive e `/content` em papéis diferentes

O Colab pode desconectar e perder o disco temporário. O Drive guarda dados selecionados e resultados; `/content` oferece leitura local durante treino e avaliação. Novos datasets são baixados em arquivos completos escolhidos, em vez de usar streaming contínuo de exemplos, porque a varredura inicial OpenFake foi lenta/dependente da conexão e muitas leituras pequenas no Drive também têm latência (política [docs/POLITICA_DADOS_FUTUROS.md]). O custo é espaço e tempo de cópia por sessão. `last.pt`, manifestos e pastas `.incompleto` permitem retomar tarefas sem reescolher imagens ou sobrescrever checkpoints. O notebook da 3.9 [Notebooks/fase_3/rodar_fase3_9_tgif_colab.ipynb] separa preparo, cópia, smoke, treino e avaliação em ações manuais; a existência da automação não significa que o treino execute sem o usuário iniciar a célula.

10. Limitações e interpretação crítica

Dependência do domínio. O próprio teste de faces da Fase 2 alcançou 99,74%, mas o mesmo checkpoint quase nunca chamou sintéticas no OpenFake com o limiar original. A discrepância é evidência de que sucesso no conjunto próprio não comprova robustez em imagens distintas. Nas fases posteriores, o melhor Mirage observado foi 62,3%, abaixo da meta externa; a última fase implementada obteve 58,75% (resultados [ESTADO_PROJETO.md]). O guia descreve o detector como um sistema implementado e avaliado, **não como um detector universalmente confiável**.

Dados e rótulos. OpenFake, Community Forensics, AIGI e TGIF têm diferentes geradores, conteúdos, resoluções e maneiras de obter fotos reais. Algumas fontes reais individuais são desconhecidas (`n/a`); o rótulo de dataset não equivale a uma certificação forense de cada imagem. Duplicatas exatas foram pesquisadas, mas a revisão visual da 3.9 encontrou quase duplicatas envolvendo calibração. Há 71 candidatos dHash no relatório V3, mas só **19 candidatos internos exibidos** foram revisados visualmente; os demais não foram declarados duplicatas (revisão visual [docs/REVISAO_VISUAL_FASE3_9.md]). Não se deve extrapolar essa amostra para uma taxa total de contaminação.

Atalhos e transformações. Formato, resolução e tamanho do arquivo estão associados ao rótulo em partes dos dados. A normalização dos pares novos reduz alguns atalhos, mas não os remove dos dados antigos. JPEG, cortes de 224 px e redimensionamento podem apagar pequenas pistas ou criar novas; a FFT também é sensível a essas operações. A AUC do baseline estatístico de tamanho na seleção interna é alerta metodológico, não prova de que o modelo principal use exatamente essa pista. Tampouco a presença de ViT, FFT e CNN prova que a rede identifique anomalias físicas, incoerência global, copigmentação ou um artefato específico. Para afirmar tal detecção seria necessário um diagnóstico específico, que não consta como resultado conclusivo dos arquivos analisados.

Falsos positivos, falsos negativos e incerteza. A 3.8 aumentou o recall Mirage para 55,7%, mas a especificidade caiu a 64,1%; a 3.9 recuperou parte da especificidade e perdeu recall. Assim, otimizar só um erro pode piorar a acurácia total. Temperatura e limiar ajustados internamente não garantem probabilidades calibradas para o domínio de fotos arbitrárias. A mensagem textual deve dizer “chance estimada”, nunca “origem comprovada” (função de linguagem [experimentos/linguagem_predicao.py]). Em imagens muito pequenas, ampliadas antes do recorte, a informação original não é recuperada pela interpolação.

Limites dos benchmarks. RAISE/MNW, Mirage e as 20 imagens antigas são conjuntos diferentes. O Mirage foi usado em várias comparações após sua primeira observação; por isso, seu resultado é descritivo para as fases posteriores, ainda que os checkpoints tenham sido congelados antes de cada inferência. A ausência de sobreposição exata não elimina imagens derivadas da mesma foto. A seleção interna que escolhe o checkpoint não deve ser promovida a teste independente. As métricas recentes vêm de saídas Colab preservadas; o Drive não foi inspecionado diretamente nesta revisão local. Nenhuma comparação apresentada isola causalmente todos os fatores de cada versão.

Recursos e produto. O treinamento e as avaliações requerem GPU Colab, armazenamento no Drive e cópia local dos dados. Uma sessão interrompida pode exigir nova cópia para `/content`; a retomada por `last.pt` vale após uma época salva. Este repositório implementa um classificador experimental e rotinas de apresentação de probabilidades, sem API de produção nem validação universal de fotos de usuários. Essas limitações definem o alcance científico do trabalho realizado.

11. Glossário técnico

Termo	Significado acessível e exemplo no TCC
Acurácia	Fração de todas as imagens acertadas. No Mirage da 3.9, 1.175/2.000 = 58,75%.
Acurácia balanceada	Média do recall sintético e da especificidade real; útil quando as classes têm tamanhos diferentes.
AdamW	Otimizador que usa médias adaptativas dos gradientes e separa o decaimento de pesos; é usado no ajuste fino.
AMP / precisão mista	Uso combinado de formatos numéricos, sobretudo `float16` em partes do cálculo, para economizar recursos de GPU.

Termo	Significado acessível e exemplo no TCC
AUC ROC	Área sob a curva que compara taxa de detecção e de falso alarme ao variar o limiar; mede ordenação de pontuações.
Backpropagation	Cálculo dos gradientes de cada peso a partir da perda; ocorre depois do forward no treino.
Batch	Pequeno lote de exemplos processado em conjunto; nas fases recentes, tamanho configurado de 32 para treino.
BatchNorm	Normalização de ativações em camadas; aparece na CNN do ramo FFT.
Calibração	Ajuste da interpretação das pontuações; no projeto, temperatura e limiar são definidos em partição interna separada.
Checkpoint	Arquivo que guarda estado do modelo e às vezes do otimizador; `last.pt`, `best.pt` e `phase3_final.pt` têm papéis diferentes.
CNN	Rede convolucional; EfficientNet-B0 e FFT-CNN são exemplos no TCC.
Convolução	Filtro aprendido aplicado localmente e repetido pela imagem ou pelo espectro.
Data augmentation	Transformação aleatória de treino, como JPEG, blur e redimensionamento, para expor o modelo a variações.
Dataset	Conjunto organizado de exemplos; OpenFake, Community Forensics, AIGI e TGIF forneceram dados internos.
dHash	Resumo perceptual simples usado para levantar pares visualmente parecidos; distância pequena é candidato, não prova de duplicação.
Dropout	Desativação aleatória de ativações durante treino, para regularização.
Early stopping	Interrupção após épocas sem melhorar o critério interno; na 3.9, paciência 3 para AUC de seleção.
EfficientNet-B0	CNN pré-treinada em ImageNet usada como terceiro ramo desde a Fase 2.
Época	Quantidade planejada de sorteios/exemplos de uma rodada de treino; com sampler com reposição, não implica ver cada arquivo uma única vez.
F1-score	Média harmônica de precisão e recall da classe positiva.
FFT	Algoritmo que calcula a Transformada de Fourier; o ramo FFT-CNN aprende sobre a magnitude logarítmica do espectro.
FN / falso negativo	Sintética chamada de real; na seleção interna da 3.9 houve 182.
FP / falso positivo	Foto real chamada de sintética; na seleção interna da 3.9 houve 59.
Gate	Rede pequena que aprende pesos de mistura para os vetores ViT, FFT e CNN.
Generalização	Capacidade de funcionar em imagens distintas das usadas para ajustar o modelo.
Gradiente	Sensibilidade da perda a uma pequena mudança de um parâmetro; orienta a atualização pelo otimizador.
Hash SHA-256	Identificador derivado dos bytes; ajuda a detectar cópias exatas e alterações, mas não imagens recomprimidas semelhantes.
Holdout	Grupo deliberadamente reservado fora do treino; na 3.2/3.3, `gpt-image-2` e `nano-banana-pro`.
Inferência	Uso de pesos já treinados para pontuar uma imagem, sem atualização de parâmetros.
Label smoothing	Treino que evita exigir confiança total no rótulo, distribuindo uma fração pequena à outra classe.
Learning rate	Taxa de aprendizado; controla o tamanho das mudanças de pesos a cada atualização.

Termo	Significado acessível e exemplo no TCC
Logit	Escore bruto da rede para uma classe; precisa passar por softmax para virar probabilidade estimada.
Manifesto	Registro dos arquivos, rótulos, divisões e hashes usados em uma execução.
Overfitting	Aprender detalhes específicos do treino sem transferir bem a imagens novas.
Patch	Região quadrada da imagem; o ViT usa patches de 16 × 16 pixels.
Piloto	Verificação de acesso, formato e viabilidade de dados, sem treino final nem métrica de acurácia.
Precisão	Fração correta entre as imagens classificadas como sintéticas; difere da acurácia total.
Probabilidade estimada	Saída de softmax após temperatura; descreve o julgamento do modelo, sem provar origem.
Recall	Fração de sintéticas existentes que foram detectadas.
Sampler	Regra de sorteio de exemplos para os batches; a 3.4 v2 usa massa de grupo proporcional a `√n`.
Scheduler	Regra que muda a taxa de aprendizado ao longo das épocas; o projeto usa warmup e decaimento cosseno.
Seleção interna	Divisão que escolhe a melhor época; seus indicadores são descritivos e não teste independente.
Smoke test	Passagem curta que confirma conexões técnicas, checkpoint, forward e escrita, sem medir acurácia.
Softmax	Operação que converte os dois logits em valores entre 0 e 1 que somam 1.
Temperatura	Escalar positivo que divide os logits antes do softmax; controla a concentração da probabilidade.
Teste externo	Imagens fora do treino e da calibração usadas para medir transferência; o Mirage já observado é comparação, não teste cego de novas decisões.
Threshold / limiar	Ponto de corte para transformar `p_sintética` em rótulo binário.
TN / verdadeiro negativo	Foto real corretamente chamada de real.
TP / verdadeiro positivo	Sintética corretamente chamada de sintética.
Transfer learning	Reaproveitamento de pesos aprendidos antes; por exemplo, EfficientNet-B0 ImageNet na Fase 2 e checkpoint 3.3 nas fases recentes.
Underfitting	Aprendizado insuficiente até para regularidades do treino.
ViT	Vision Transformer que usa patches e atenção para relacionar regiões; o ramo do TCC devolve o token `[CLS]`.
Weight decay	Penalização/controle do crescimento dos pesos na atualização AdamW.

12. Revisão e fixação do conhecimento

12.1. Conceitos e funcionamento

1. Qual é a pergunta que o detector responde? Uma foto real de um cenário encenado é “falsa”?

Ele estima se a **imagem** pertence à classe fotográfica real ou à classe gerada/alterada por IA. Não avalia a verdade do evento retratado. Uma foto de cenário encenado ainda foi capturada por câmera e, pela definição do projeto, continua real; uma cena plausível gerada por IA continua sintética. A classe é operacional e vem de rótulos de dataset, não de uma prova automática de procedência.

2. O que diferencia inteligência artificial, aprendizado de máquina e aprendizado profundo neste trabalho?

IA é o campo abrangente. O detector é aprendizado de máquina porque seus parâmetros são ajustados por exemplos rotulados. É aprendizado profundo porque usa ViT, CNNs e várias camadas treináveis para extrair características diretamente de pixels. A aplicação dessas redes a imagens o coloca em visão computacional.

3. Qual é a diferença entre treinar e fazer inferência?

No treino, o forward gera logits, a perda compara esses logits ao rótulo, o backward calcula gradientes e o otimizador atualiza pesos. Na inferência, os pesos são carregados do checkpoint, o modelo é colocado em modo de avaliação, calcula logits/probabilidades e **não** chama backward nem atualiza parâmetros. O smoke executa um forward curto para validar a infraestrutura; ele não é um treino nem uma avaliação de desempenho.

4. Por que uma época com sampler com reposição não significa "cada imagem foi vista uma vez"?

O `WeightedRandomSampler` sorteia índices de acordo com pesos e `replacement=True`. Cada sorteio devolve o índice à lista de possibilidades. Uma imagem pode repetir no mesmo ciclo e outra pode não ser sorteada. "Época" indica o número planejado de sorteios equivalente ao tamanho do treino, não uma visita determinística a todos os arquivos.

5. O que é o token `[CLS]` do ViT e como a atenção o atualiza?

É um vetor treinável acrescentado à sequência dos 196 patches. Os 12 blocos de atenção relacionam esse token aos demais e permitem que seu vetor final resuma informação da imagem para a fusão. Queries e keys determinam pesos de associação; values são combinados por esses pesos. O `[CLS]` final é uma representação aprendida, não uma legenda semântica explicável por si.

6. O que a FFT acrescenta e por que ela não prova que a foto é de IA?

A FFT representa variações espaciais em frequência e direção. O ramo usa magnitude deslocada e comprimida por `log1p`, depois uma pequena CNN aprende características nesse espectro. Padrões periódicos ou texturas podem fornecer informação complementar aos pixels. Porém fotos reais, compressão e redimensionamento também alteram frequências; o ramo não fornece um selo de origem nem identifica necessariamente um artefato específico.

7. Por que usar EfficientNet-B0 depois de já haver ViT e FFT?

O código da Fase 2 adiciona uma CNN com pesos ImageNet para obter características espaciais hierárquicas e locais. Ela pode complementar as relações entre patches do ViT e a representação frequencial. O gate aprende a misturar os três vetores. O desempenho melhor no teste próprio da Fase 2 ocorreu junto com várias mudanças; portanto, não quantifica isoladamente a contribuição do EfficientNet.

8. Arquitetura, pesos e checkpoint são sinônimos?

Não. Arquitetura é a estrutura de camadas. Pesos são números aprendidos nessa estrutura. Checkpoint é um arquivo que armazena esses números e possivelmente configuração, calibração, estado do otimizador e época. Na 3.9, a arquitetura é a `Phase2Detector`, os pesos iniciais vieram da 3.3 e o arquivo final `phase3_final.pt` contém pesos escolhidos mais temperatura e limiar próprios da 3.9.

9. O que fazem label smoothing, warmup, clipping e AMP no laço de treino?

Label smoothing suaviza a exigência de confiança absoluta no rótulo. Warmup aumenta gradualmente o learning rate no início; o cosseno o reduz depois. Gradient clipping limita atualizações oriundas de gradientes muito grandes. AMP usa precisão numérica mista para reduzir custo de GPU, com `GradScaler` protegendo gradientes pequenos. São técnicas implementadas, mas não há ablação que permita atribuir um ganho externo isolado a cada uma.

10. Por que `best.pt` e `last.pt` não são intercambiáveis?

`best.pt` guarda a melhor época pelo critério da seleção, usada para o modelo final. `last.pt` guarda o estado da época mais recente, incluindo otimizador e scheduler para retomar uma sessão interrompida. A última época pode ter AUC menor que a melhor. `phase3_final.pt` contém os melhores pesos já acompanhados de temperatura e limiar calibrados.

12.2. Dados, avaliação e interpretação

11. Qual foi a mudança essencial da Fase 1 para a 2?

A Fase 1 unia ViT-Tiny e FFT-CNN por concatenação. A Fase 2 adicionou EfficientNet-B0 pré-treinada, projetou os três vetores para 256 componentes e passou a combiná-los com gate. Também adotou duas etapas de treinamento. O teste próprio ficou alto, mas a referência posterior OpenFake mostrou transferência muito menor; isso demonstra por que é preciso informar sempre o conjunto avaliado.

12. O que diferenciou 3.1, 3.2 e 3.3?

A 3.1 iniciou o ajuste fino multigerador no OpenFake; na execução registrada, o teste não continha os grupos holdout pretendidos. A 3.2 reservou `gpt-image-2` e `nano-banana-pro` fora do treino, confirmou 155 imagens desses grupos no teste e aumentou a probabilidade de recompressão JPEG aleatória do treino de 0,3 para 1,0. A 3.3 manteve as transformações e o laço de treino da 3.2 em código, registrou outra execução e acrescentou auditorias no notebook. A arquitetura de três ramos permaneceu. As diferenças de métricas não isolam causalmente cada mudança porque protocolos e amostras também diferiram.

13. Por que a v2 da 3.4 não permite afirmar que apenas o sampler melhorou a acurácia?

A v2 mudou a massa dos grupos para `√n` e observou 72,88% contra 71,55% no OpenFake. Porém desativou o replay efêmero da Fase 2, cuja seleção na v1 não foi persistida; não há garantia de que os dois treinos diferiram **somente** no sampler. O resultado comprova a diferença observada nesse teste, não o efeito causal único da fórmula de amostragem.

14. O que os pares AIGI e TGIF controlam, e o que não controlam?

Cada par conecta uma real e uma sintética por UID/ID e mantém ambas na mesma divisão. Processar as duas classes para JPEG e lado curto 256 diminui um atalho simples dentro desses dados novos. Não iguala todos os conteúdos visuais, não corrige automaticamente a associação de formato/resolução nos dados históricos e não prova que o modelo passou a usar sinais forenses em vez de pistas de dataset.

15. Se um detector obtém AUC 0,69, sua acurácia precisa ser 69%?

Não. AUC avalia a ordenação das pontuações ao longo de muitos limiares; acurácia conta as decisões num limiar específico. No Mirage da 3.7, os valores foram AUC 0,6908 e acurácia 62,3%. Na Fase 2 do RAISE/MNW, AUC 0,6426 coexistiu com acurácia 50% e recall zero, pois o limiar congelado praticamente nunca chamava uma imagem de sintética.

16. Temperatura 0,44 é “baixa demais”? Limiar 0,47 é “ruim” por não estar perto de 1?

Não existe objetivo de aproximar ambos de 1. Temperatura menor que 1 torna o softmax mais concentrado para os logits daquele modelo; ela deve ser avaliada pela calibração, não por proximidade a um número bonito. Limiar define o compromisso entre FP e FN: elevá-lo geralmente reduz acusações de fotos reais e também perde sintéticas. Os valores são escolhidos em dados internos. Modelos diferentes produzem distribuições de logits diferentes; comparar apenas os números T e limiar entre eles não ordena sua qualidade.

17. Qual é a diferença entre precisão, recall e acurácia usando a 3.8 no Mirage?

O recall 55,7% significa que 557 das 1.000 sintéticas foram identificadas; 443 passaram como reais. A especificidade 64,1% significa que 641 das 1.000 reais foram preservadas; 359 viraram falsos alarmes. A acurácia é `(557+641)/2.000 = 59,9%`. A precisão é `557/(557+359) ≈ 60,8%`: entre as chamadas sintéticas, essa fração era sintética. Cada medida responde a pergunta distinta.

18. Por que os 85,62% da seleção interna da 3.9 não demonstram a meta do TCC?

Porque a seleção de 1.676 imagens participou da escolha do melhor checkpoint. Seus 1.435 acertos são um diagnóstico dentro do processo de desenvolvimento. No Mirage, com 2.000 imagens distintas e checkpoint congelado, a mesma fase teve 58,75%. A meta exige acurácia externa independente; não basta uma métrica interna numericamente superior a 85%.

19. O que a revisão de duplicatas da 3.9 realmente encontrou?

Hashes exatos não localizaram cópias binárias entre divisões internas; a revisão visual de 19 candidatos exibidos encontrou duas fotografias quase iguais cruzando **seleção × calibração** e **treino × calibração**, além de uma cena sintética relacionada entre treino e calibração. Isso enfraquece a afirmação de separação visual completa e exige cautela na leitura de T/limiar. Não mede quantos pontos de acurácia foram afetados, nem prova vazamento entre treino e seleção nos pares inspecionados; os demais candidatos não foram integralmente revisados.

20. Qual conclusão técnica defensável sobre a última versão e sobre a melhor observada?

A Fase 3.9 é a **mais recente implementada**: acrescentou pares TGIF, foi treinada, calibrada, avaliada e auditada. Ela não é a melhor no Mirage: teve 58,75% de acurácia, frente aos 62,3% da 3.7 no mesmo benchmark. A 3.9 mostrou dificuldade interna concentrada nas sintéticas TGIF, e a revisão visual revelou limitação de separação envolvendo calibração. Esses fatos descrevem o experimento; não provam uma causa única da queda nem que o detector satisfaça a meta de 85% em imagens externas independentes.